# Collaborative Filtering for Tool Selection in LLM Agents

Исследовательский proposal, курс «Рекомендательные системы», октябрь 2026

**Команда:** Илья Звездин, Святослав Полонский, Артём Туревич **Репозиторий:** https://github.com/pururus/CFTSagent/

## 1. Задача, цель и гипотезы

**Вход:** текстовый запрос $q$ и каталог API-инструментов $T$ (название, описание, схема параметров). **Выход:** ранжированный набор $S_K(q) \subseteq T$ из $K$ API, который увидит LLM-агент. Каталоги из тысяч API не помещаются в контекст, а длинный список ухудшает выбор инструментов самой LLM, поэтому нужен предварительный отбор top-$K$. В терминах рекомендательных систем: «пользователь» — запрос, «item» — API. Нетривиальность. (i) Почти каждый запрос новый — user cold start, CF в чистом виде неприменим. (ii) Запрос может требовать несколько *взаимодополняющих* API, тогда как top по cosine часто содержит дубли одной функции. (iii) Разметка неполна: неразмеченный API не обязательно нерелевантен.

**H1:** история использования инструментов в похожих запросах добавляет информацию к текстовому сходству запроса и описания API. **H2:** эффект CF сильнее для multi-tool задач и новых *комбинаций* уже известных инструментов. **H3:** на естественных запросах (ToolLens), которые не цитируют названия API, относительный вклад CF выше, чем на синтетическом ToolBench. На разнородных каталогах (ToolRet) выигрыш сохраняется только для API с историей взаимодействий. **H4:** рост полноты retrieval повышает task success фиксированного агента, но рост NDCG не обязан ему соответствовать.

## 2. Существующие решения и наш вклад

**ToolLLM/ToolBench** [1] даёт API-каталог RapidAPI, инструкции и разметку релевантных API. На нём обучен агент ToolLLaMA (LLaMA-2-7B, SFT на траекториях ChatGPT, найденных поиском DFSDT) с dense retriever на BERT-base, качество оценивается ToolEval (pass rate и win rate с ChatGPT-судьёй) и сопоставимо с ChatGPT. **COLT** [2] уже сочетает семантическое обучение с graph collaborative learning (query–scene–tool) и вводит датасет **ToolLens** с естественными multi-tool запросами. **ToolRet** [3] объединяет 35 датасетов в бенчмарк tool retrieval и показывает, что сильные IR-модели на нём заметно проседают. **Наша работа.** Реализуем простые *индуктивные* CF-гибриды (KNN по запросам, co-occurrence инструментов), работающие для новых запросов без переобучения. Измеряем их добавочную ценность относительно необученного *и* обученного semantic retriever на трёх бенчмарках с разной «естественностью» запросов и разным каталогом. Отдельно оцениваем in-domain, перенос и item cold start.

## 3. Метод

Item — **API endpoint**, а не сервис целиком. На train строим разреженную матрицу $R_{it} = 1$, если API $t$ размечен как релевантный запросу $q_i$. Неразмеченные пары считаем *неизвестными*.

1. **Semantic:** замороженный `bge-small-en-v1.5` [5] кодирует запрос и текст API, $s_{\text{sem}}(q,t) = \cos(e_q, e_t)$, плюс лексический BM25.
2. **Query-KNN CF** (решение user cold start): для нового $q$ берём $m$ ближайших *train*-запросов по эмбеддингам: $s_{\text{CF}}(q,t) = \sum_{i \in N_m(q)} w(q,q_i)\, R_{it}$, где $w$ — нормированные неотрицательные веса сходства.
3. **Гибрид:** $s(q,t) = \alpha\, \widetilde{s}_{\text{sem}}(q,t) + (1-\alpha)\, \widetilde{s}_{\text{CF}}(q,t)$. Для API без истории $s_{\text{CF}} = 0$ и модель откатывается к semantic. Как абляцию пробуем сглаживание CF-оценки по соседям API в пространстве эмбеддингов (item cold start в ToolRet).
4. **Co-occurrence reranking:** жадно дополняем набор $S$ с бонусом $\beta \max_{t' \in S} \widetilde{C}_{tt'}$, где $\widetilde{C}$ — нормированная (cosine/Jaccard) train-матрица $C = R^\top R$. Вариант со штрафом за дублирование проверяет H2.
5. **Обучаемый контроль:** Two-Tower — линейные проекции $u_q = W_q e_q$, $v_t = W_t e_t$ поверх замороженных эмбеддингов, BPR: $\mathcal{L} = -\sum_{(q,t^+,t^-)} \log \sigma(u_q^\top v_{t^+} - u_q^\top v_{t^-})$, негативы — случайные и hard из semantic top-20. Нужен, чтобы отделить эффект CF от обычной supervised-адаптации к датасету.

## 4. Данные и протокол оценки

| Датасет | Запросы | Каталог | Роль и статус проверки |
|---|---|---|---|
| ToolBench [1] | ∼126k | 16 464 API | Основной train/test, группы G1 (single-tool), G2/G3 (multi-tool). Поля `query`, `query_id`, `api_list` и скрипт подготовки retriever-данных [7] проверены по коду. |
| ToolLens [2] | 18 770 | 464 API | Естественные multi-tool запросы, ≈2.7 API на запрос, API из каталога ToolBench. Файлы скачаны и проверены: 16 893 / 1 877 запросов train/test, запросы не пересекаются, все test-API есть в train. |
| ToolRet [3] | 7 961 test | 44 453 | 35 подвыборок (ToolBench, Gorilla, API-Bank, MetaTool, …) + ToolRet-train (208 826 примеров) на HuggingFace. В train позитивы даны текстом, и их нужно сопоставить с ID каталога. |

**Протоколы.**

- **A** — ToolBench, split по запросам 80/10/10 с дедупликацией близких формулировок, CF строится только из train. Дополнительно откладываем *комбинации* API, оставляя каждый API известным по train.
- **B** — ToolLens, официальный test. В официальном репозитории `tune` совпадает с `test`, поэтому validation

выделяем из train.

- **C** — перенос с ToolBench-train на ToolLens-test (сопоставление API по `tool_name/api_name`): переносится ли CF с синтетических запросов на естественные.
- **D** — ToolRet: обучение на ToolRet-train, отчёт по подвыборкам с разделением на «тёплые» (API встречались в train) и «холодные». Подвыборки ToolBench/ToolLens проверяем на пересечение с нашими train-данными.

Поиск всегда идёт по *полному* каталогу, а не по `api_list` данного запроса. **Метрики:** Recall@K, NDCG@K, FullCoverage@K (все размеченные API в top-$K$), $K \in \{3, 5, 10, 20\}$, отдельно по G1/G2/G3 и подвыборкам, с bootstrap-интервалами по запросам. Метрики считаются относительно *аннотированных* API, что не исключает альтернативных решений.

**Агент (расширение):** 100–200 задач G2/G3 на фиксированном `Qwen3-4B-Instruct-2507` [6] с наборами Oracle / Semantic / Hybrid top-$K$ и виртуальными API StableToolBench [4]. Метрики — task success, число вызовов, токены, зависимость от $K$.

## 5. Бейзлайны и план экспериментов

Бейзлайны: **Popularity** (сколько даёт популярность API), **BM25** (лексика, должен быть силён на ToolBench, где запросы часто называют API), **zero-shot BGE** (семантика без обучения)

Дальнейшие эксперименты: **query-KNN CF** (только коллаборативный сигнал), **BGE + KNN-CF**, **BGE + co-occurrence**, **Two-Tower** и **Two-Tower + CF** (отделяют CF от supervised-обучения). Абляции: параметры $m$, $\alpha$, $\beta$, доля train-истории 1/5/20/100% (сколько истории нужно CF), G1 vs G2/G3, seen vs unseen комбинации, ToolBench vs ToolLens (H3), тёплые vs холодные API в ToolRet, слабый vs сильный энкодер (`bge-small` vs `bge-base`).

## 6. Этапы, команда, ресурсы и риски

| Контрольная точка | Результат и ответственность |
|---|---|
| **19 ноября** (прогресс 1) | Илья Звездин: загрузка ToolBench и ToolLens, общий формат, splits, проверка утечек, метрики. Святослав Полонский: Popularity, BM25, zero-shot BGE на обоих датасетах. |
| **3 декабря** (прогресс 2) | Святослав Полонский: KNN-CF, гибрид, подбор параметров, протокол C. Артём Туревич: co-occurrence, Two-Tower, абляции. Илья Звездин: ToolRet (протокол D). |
| **9 декабря** (финал) | Артём Туревич: графики, анализ ошибок, agent-пилот. Все: отчёт, README с инструкциями воспроизведения. |
| **10–11 декабря** (защита) | Все: презентация, ответы на вопросы. |

**Ресурсы.** KNN-CF, co-occurrence и метрики — CPU и разреженные матрицы. Эмбеддинги ~400k текстов `bge-small` — минуты на GPU 8–16 GB (или Colab/Kaggle), Two-Tower обучается там же за минуты. Agent-пилот с 4B-моделью: ≥16 GB VRAM и vLLM либо внешний inference.

**Риски:** синтетичность и утечки в ToolBench (смягчаем дедупликацией и ToolLens), неполная разметка (task success), нестабильные внешние API (StableToolBench), трудоёмкое сопоставление ToolRet-train с каталогом (протокол D — расширение).

**Минимальный результат:** воспроизводимое offline-сравнение Popularity, BM25, BGE, KNN-CF и гибрида на ToolBench и ToolLens с проверенными splits, абляциями по G1/G2/G3 и объёму истории. ToolRet и агент — расширения.

## Источники


[1] Qin Y. et al. ToolLLM: Facilitating Large Language Models to Master 16000+ Real-world APIs. ICLR 2024. arXiv:2307.16789, https://github.com/OpenBMB/ToolBench.

[2] Qu C. et al. Towards Completeness-Oriented Tool Retrieval for Large Language Models. CIKM 2024. arXiv:2405.16089, ToolLens: https://github.com/quchangle1/COLT.

[3] Shi Z. et al. Retrieval Models Aren't Tool-Savvy: Benchmarking Tool Retrieval for Large Language Models. Findings of ACL 2025. arXiv:2503.01763, https://huggingface.co/collections/mangopy/tool-retrieval.

[4] Guo Z. et al. StableToolBench: Towards Stable Large-Scale Benchmarking on Tool Learning of LLMs. Findings of ACL 2024. https://github.com/THUNLP-MT/StableToolBench.

[5] BAAI. bge-small-en-v1.5. https://huggingface.co/BAAI/bge-small-en-v1.5.

[6] Qwen Team. Qwen3-4B-Instruct-2507. https://huggingface.co/Qwen/Qwen3-4B-Instruct-2507.

[7] ToolBench retriever preprocessing. https://github.com/OpenBMB/ToolBench/blob/master/preprocess/preprocess_retriever_data.py.